Intelligent Hot Desk Management System

May 2026

Olmo Combley Lopez
Department of Computer Science
University of Oxford
olmo.combley@wadham.ox.ac.uk

Marc Denis
Department of Computer Science
University of Oxford
marc.denis@worc.ox.ac.uk

Joseph Edwards
Department of Computer Science
University of Oxford
joseph.edwards@new.ox.ac.uk

Isaac Owen
Department of Computer Science
University of Oxford
isaac.owen@lmh.ox.ac.uk

Krish Sen
Department of Computer Science
University of Oxford
krish.sen@hertford.ox.ac.uk

Moncef Slimani
Department of Computer Science
University of Oxford
moncef.slimani@magd.ox.ac.uk

***Abstract*—This report describes the design, implementation, and evaluation of a hot desk management software. The system provides a web interface for booking desks, an algorithm for recommending suitable desks, and various administrative tools. The final implementation combines a React frontend, a Go REST API, and a PostgreSQL database. The project met its core functional goals and produced a maintainable platform for future extensions.**

I. Overview

Hybrid working has changed how many organisations use office space. Permanent desk assignment can leave desks unused on remote work days, while unmanaged hot desks can make it difficult for employees to sit near colleagues, plan attendance, and avoid conflicts over limited space. Kuehne + Nagel asked for a system that could support practical hot desk booking while also improving the operational usefulness of desk choices.

Our project comprises a full-stack web application for managing desks, floors, users, teams, and bookings. Employees can browse available desks on an interactive floor plan, and reserve a desk for a selected time interval. Administrators can manage data and oversee bookings. The system includes a recommendation algorithm that scores candidate desks based on their proximity to colleagues both on the same team, and on related teams.

II. Project Aims

Our principal aims were:

- to provide employees with a fast and understandable method of locating and reserving a desk;
- to provide administrators with control over floors, desks, users, and booking oversight;
- to provide team-aware recommendations so that desk allocation fosters effective collaboration;
- to preserve booking correctness under concurrent use;
- to achieve the above within a modular codebase that can be extended by later developers.

III. Plan

During the initial planning phase, we tried to identify areas that could pose problems, so that we could give them extra attention during development.

A. Office visualisation

We knew that we wanted to provide users with an intuitive visualisation of offices during the booking process. However, we were unsure of the best way to implement this, balancing technical complexity with the desire for a seamless user experience.

B. Data consistency

One of the project's key aims was to preserve booking correctness under concurrent use. We would need to handle data across three different components (frontend, backend and database), each with their own formats and encodings. We were keen to establish a solid system for data handling early on, hoping to thereby avoid major headaches later in development.

C. Quality of recommendations

The feature that sets our project apart from existing solutions is the desk recommendation algorithm. We wanted to design a sophisticated algorithm that would provide intelligent insights. However, we worried that there would be insufficient data for this to be feasible.

IV. Specification

After discussion with Kuehne + Nagel, we settled on the following specification.

A. Features

The application will provide the following features:

- A visual floor plan indicating available desks; users can click/tap a desk to book it
- A score for each desk based on its distance to colleagues in the same team, as well as how valuable it would have been to other teams

- The ability to view, modify, and cancel bookings
- An admin interface consisting of:
  - An interface for creating new floors based on an uploaded floor plan, as well as the option to delete a floor
  - Facilities for assigning users to teams, and creating and deleting the teams themselves
  - An interface for managing user permissions

B. Roles

The application will have different modes of interaction depending on the end user.

1) Regular user: Regular users will be able to:

- Create new bookings
- View and manage their own bookings

2) Administrator: Admins have access to additional features, notably:

- Modification of bookings they did not create
- Permission management for all users
- Team assignment
- Management of floors and floor layouts

C. Security

- Login authentication
- Robust role-based access control

D. Performance

- Support for many concurrent users
- Real-time updates with low latency

E. System Architecture

The final system follows a conventional three-tier web architecture. A React frontend communicates with a Go REST API, which in turn accesses a PostgreSQL database.

V. Methodology

The methodology developed in phases along with the project, landing on an effective and sustainable system described below.

A. Work Structure

A working pattern was quickly discussed and agreed upon, to facilitate the start of the development pipeline.

1) Development Approach: The project adopted an Agile development methodology, in order to support iterative development and continuous improvement. We chose this approach due to its flexibility: it allowed for quick changes, which was beneficial in the short timespan.

2) Project Planning and Management: The project was managed via a structured management process. We held initial team meetings to assign tasks and areas of the project to each of us.

Communication was active throughout the project, allowing for the testing and verification of other members' code and the flagging of issues. GitHub pull requests, GitHub issues, and social channels such as WhatsApp were used to great effect for this purpose. Additional meetings were held to track progress, evaluate backlog, and redistribute tasks where needed.

B. System Development

We initially focused on building a scalable structure for the application, before moving on to implement user-facing features.

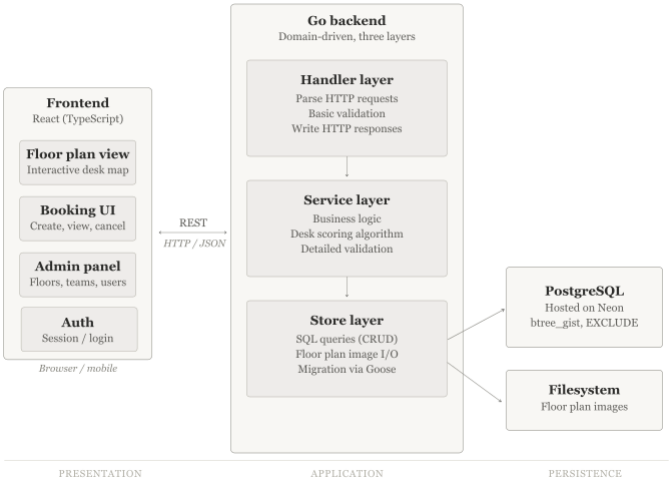

Fig. 1. High-level system architecture

1) Database: The database was implemented using PostgreSQL and hosted on Neon. We used PostgreSQL because it provides strong relational modelling, transaction support, and advanced constraint functionality, which are important to maintain booking consistency and prevent invalid desk reservations. Neon was chosen to host the database because it provided managed PostgreSQL hosting and allowed us to share a single remotely-hosted database during development.

The database schema was designed using a relational structure to separate core entities (departments, teams, users, floors, desks, bookings, and walls[1]) into individual tables. This reduced data duplication and improved maintainability by ensuring that each table represented a single logical concept. For example, teams were linked to departments through foreign keys, and users were linked to teams, resulting in a clear organisational hierarchy.

The `desks` table stores positional coordinates for each desk. This is needed both for adjacency calculations and for displaying the floor plan on the frontend. The table includes an `is_enabled` field so that administrators can temporarily disable desks without having to permanently delete their records from the database.

The `bookings` table forms the core transactional component of the system. Each booking links a user to a desk over a specified time interval using foreign key relationships. Timestamps are stored using `TIMESTAMPTZ` so that booking times are handled in a timezone-aware manner. This table uses PostgreSQL's `EXCLUDE` constraint with the `btree_gist` extension to prevent overlapping booking intervals from being

[1]We included walls to facilitate a more advanced distance metric, which has not yet been implemented; the table remains unused in the database.

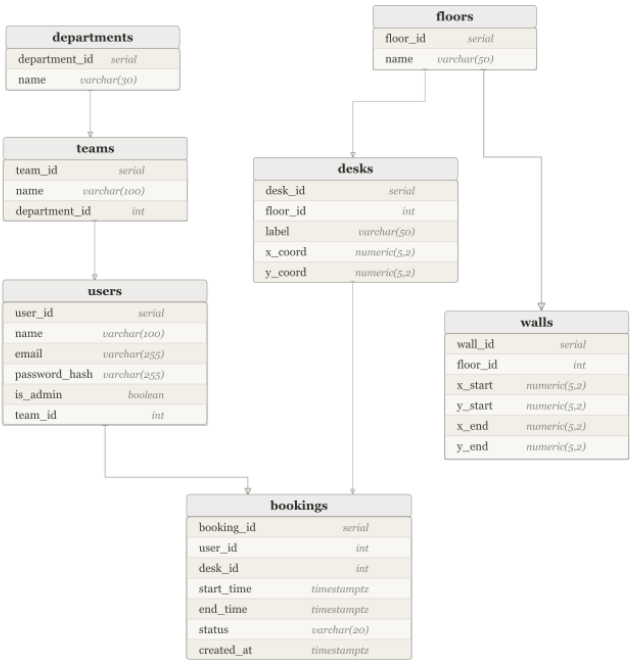

Fig. 2. Database structure

created for the same desk, ensuring that desks cannot be double-booked for the same time period.

We managed database schema changes and index creation using Goose migrations. This allowed database updates to be version-controlled and applied consistently across development environments.

In addition to the relational database hosted by Neon, we also store a floor plan image for each floor on the server's filesystem. Although it fragments our data store, we chose this approach because it keeps the database lightweight and makes the system more scalable.

2) Back end: A domain-driven architecture separates the back end into three layers to cleanly isolate concerns within each domain:

- The handler layer bridges the gap between HTTP requests and the internal backend representation. It accepts and parses the requests' query parameters from clients, performing basic validation when required arguments are not present. It subsequently calls the service layer, and writes an HTTP response to the client's request.
- The service layer acts on the internal representation of data and holds the business logic. It accepts the request arguments from the handler layer, performs more detailed validation, makes calls to the store layer, then returns a result to the handler. The service layer is responsible for any actual computation required to service requests, such as the desk usage prediction algorithm.
- The store bridges the gap between the internal representation and the database. Each function within this layer relates directly to an operation on the data store (e.g. `create`, `update`). The store layer is also responsible for saving, retrieving and deleting floor plan images on the server's filesystem.

This modular approach was extremely helpful during development because each domain is handled similarly (particularly in the handler and store layers), enabling code reuse and making the backend easy to augment. We chose Go for the implementation, because of its lightweight concurrency and fast compilation.

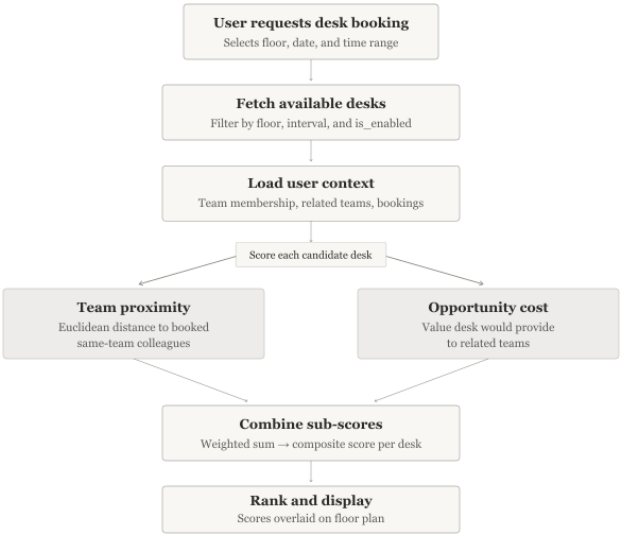

Fig. 3. Desk recommendation algorithm

3) Desk Recommendation Algorithm: The recommender scores each free desk using a composite metric that balances proximity to the requesting user's team against the disruption imposed on other teams.

For a given booking window, the system first filters for desks that are entirely unbooked during that period. Each candidate desk is then assigned a score with two components. The first is a proximity gain: the difference between the average distance from the candidate desk to the requesting user's coworkers and the minimum such average achievable across all free desks. Coworker bookings that only partially overlap the requested window contribute proportionally to this average. The second component is a displacement penalty: if the candidate desk is currently the best-ranked desk for some other team, assigning it to the requesting user forces that team to fall back to their second-best option. The penalty is taken as the largest such degradation across all other teams. The two components are summed to produce a final score. This formulation ensures that the algorithm does not consistently recommend desks that are disproportionately important to other teams, encouraging teams to book desks close to one another.

4) Authentication and authorization: The application uses cookie-based session authentication and role-based access

control. Authentication is handled by the backend, whereby the server is responsible for enforcing which operations are permitted.

Users authenticate through the `/api/auth/login` endpoint using an email address and password. During login, the submitted password is compared against the stored password hash. If the credentials are valid, the server creates a random session token, stores only its hash in the `auth_sessions` table, and returns the raw token to the browser in an HTTP-only cookie. Consequently, a database leak would not directly expose usable session tokens.

Each later request includes the session cookie automatically. The middleware reads the cookie, hashes the supplied token, and looks it up in `auth_sessions`. If the session exists, has not expired, and has not been revoked, the corresponding user is attached to the request context as an actor. Handlers and services can then make decisions using this actor, particularly for authorisation.

Regular users are restricted to ordinary booking workflows and their own user-specific data. Administrators can access management functionality such as creating desks, creating floors, updating users, and viewing broader organisational data. Route middleware protects admin-only endpoints, while service-layer methods enforce actor-specific rules such as preventing a regular user from viewing or modifying another user's private data.

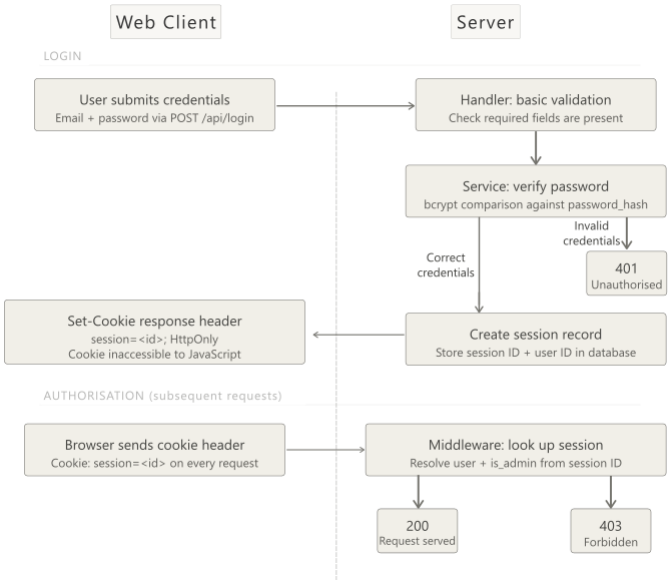

Fig. 4. Authentification process

5) API: The API used a RESTful structure, split into core domains with routes following a consistent `/api/{domain}/{id}` pattern. `PATCH` semantics were adopted for partial updates such as cancelling bookings, toggling desk availability, and updating user roles, in preference to full `PUT` replacements.

6) Frontend: The frontend is a single-page application built with React and TypeScript, communicating with the Go REST API over HTTPS. It is structured around four main pages rendered within a shared layout component that handles navigation and user context:

- Booking page: browse and reserve desks on an interactive floor plan
- My Bookings page: view and cancel personal bookings
- Admin panel: manage desks, bookings, users, and view occupancy data
- Shared layout: navigation bar, user selector, and route guards

a) Booking page: Employees select a floor and time interval, then view desk availability overlaid on an interactive floor plan. Each desk is rendered at its stored coordinates and coloured by its availability status for the selected interval. Selecting an available desk triggers a confirmation flow that calls `POST /api/bookings`. Touch support is included: a first tap reveals the desk tooltip, and a second tap confirms the booking, mirroring the hover-then-click behaviour on desktop.

b) My Bookings page: Bookings are fetched from `/api/bookings?userId={id}` and displayed as cards showing the floor, desk identifier, and time interval. Each card includes a cancel button that issues `PATCH /api/bookings/{id}` and optimistically removes the card from local state.

c) Admin panel: The admin section at `/admin` is split into four tabs rendered inside `AdminLayout`, which enforces the admin guard before mounting any child page:

- Dashboard: displays today's confirmed booking count, tomorrow's predicted bookings via `/api/bookings/predict`, total enabled desks, and per-floor occupancy as progress bars
- Desks: shows a per-floor table of desks with their enabled/disabled status; clicking Enable or Disable calls `PATCH /api/desks/{id}` and optimistically updates both the row and the enabled desk count in the header
- Bookings: paginated table of all bookings with inline controls for modification
- Users: paginated table of all users with inline controls for modification

d) Responsive design and mobile support: The layout was extended to support smaller screens with the following changes:

- The desktop nav collapses into a hamburger menu; tapping it reveals a stacked panel with navigation links, the user selector, and a logout option, all of which dismiss the panel on interaction
- Booking page filter controls use `flex-wrap` to reflow onto multiple lines on narrow viewports
- Admin tables are wrapped in horizontally scrollable containers with minimum widths calibrated to their column counts, preventing content compression
- The floor plan canvas uses `touch-none` styling and a `touchend` handler to correctly translate touch coordinates into desk selection events, while suppressing

the synthetic click that would otherwise cause duplicate placements
- Booking cards apply `min-w-0` and `truncate` to date and time text, so long strings clip cleanly rather than pushing the cancel button off screen

VI. Testing

A. Backend Testing

Our backend tests focused on domain logic and security-sensitive middleware. Unit tests covered password/session behaviour where applicable, authentication middleware, and lower-level recommendation or validation logic.

B. Frontend Testing

Frontend testing was primarily manual and integration-oriented. We frequently exercised the main user workflows in the browser: selecting a floor, choosing a date and time range, viewing desk availability, creating a booking, cancelling a booking, and using administrative floor creation. Visual testing was particularly important for the floor plan interface, because correctness depends on the user seeing desk positions and availability states clearly.

C. Database Testing

Database testing concentrated on migration correctness and booking integrity. We checked the exclusion constraint on bookings by attempting to create overlapping reservations. This ensured that our database would reject invalid state even if application-level validation missed a case.

VII. Plan evaluation

The initial plan correctly identified the main technical risks: interactive floor plans, data consistency, and recommendation quality. We benefited tremendously from implementing the database and API early, as this established stable interfaces for the frontend. While the recommendation algorithm could be enhanced given more time, we believe that it is sophisticated enough to be provide genuinely value to end users, particularly for large workspaces where inspecting all current bookings manually would be unfeasible.

VIII. Specification review

The specification was mostly met to our (and Kuehne + Nagel's) satisfaction; Table I shows a breakdown of the key elements. We managed to maintain data integrity using database-level constraints, and structured the codebase in a maintainable fashion, with clear API boundaries and migration-based schema management.

Some parts of the specification were too ambitious for the available time: real-time updates and wall-aware distance calculation would both have been valuable features, but they were less essential than a reliable core. Our final project therefore prioritises correctness and maintainability over breadth, while successfully meeting most of the specification.

The specification failed to anticipate the need to store floor plan images, in order for them to be displayed on the booking page. This added some complexity to the way data is stored, but did not cause any problems during development, for example with deadlines or time estimates.

TABLE I
Evaluation Against Core Requirements

Requirement	Status	Evidence
Desk booking	Met	Users can create and cancel bookings through the API and frontend.
Availability display	Met	Floor plans show desk state for the selected time interval.
Admin management	Met	Admin workflows exist for floors, desks, and organisational data;.
Recommendation	Met	Desk scores are computed from team proximity and opportunity cost.
Booking integrity	Met	Application validation is backed by PostgreSQL constraints.
Real-time updates	Partial	State refreshes after user actions, but push-based live updates were not implemented.

IX. Failures

A. Timestamp handling issues

Despite our concerns for data consistency, during development we encountered an issue with the handling of booking timestamps. Initially, the database used PostgreSQL's standard `TIMESTAMP` type for constraints and different components of the application also handled time zone conversion inconsistently, with some values being converted to UTC while others were not. Although bookings were stored and validated correctly in the backend, the frontend desk selection view would display desks as being booked an hour earlier[2] than they actually were. The issue was resolved by standardising time zone handling across the application, and using the `TIMESTAMPTZ` type in the database schema instead.

B. More advanced recommendation algorithm

When designing the specification, we intended for the recommendation algorithm to account for obstacles in the office. That is, desks might appear to be close on the floor plan despite being separated by a wall, in which case the algorithm should not consider them close. Our idea was that in addition to marking desk locations, admins would mark obstacles/walls on the floor plan when creating a floor; then we would use a more advanced distance metric (or a pathfinding algorithm) to compute distances between desks. Implementing this would have made our product more sophisticated and helpful to users; but given the limited time frame of the project, we decided to forgo it in favour of more essential components.

X. Responsible innovation

The classical model of assigning a desk per employee has become largely obsolete with recent technological advancements. Our product empowers companies to efficiently

[2]Corresponding to the one-hour difference between UTC and BST

manage their office space, reducing overheads and ultimately contributing to greater financial and technological growth.

It has been argued that employees are, in fact, less productive when working remotely and that the hot desk model is to be discouraged. We believe that different working models are suitable in different contexts, and that our product is a valuable tool for scenarios where hot desks make sense.

XI. Ethical considerations

The system processes personal and workplace data, including user identity, team membership, and booking history, incurring privacy responsibilities. Booking data should be visible only where operationally justified, and administrative access ought to be limited to appropriate staff.

The recommendation algorithm should also be presented carefully. It should assist users rather than silently controlling where they sit. Users should retain agency to choose another available desk, and administrators should avoid using booking data for unnecessary surveillance of attendance patterns.

XII. Future work

Future work on the project could involve:

- Aforementioned enhancements to the recommendation algorithm
- A fully complete set of administrative tools
- Further polish to the UI, especially across varying screen resolutions
- A dedicated app for mobile devices

Acknowledgments

We would like to thank our external supervisor, Chris Ngan, for his guidance, industry insight, and continued support throughout the project. His feedback helped shape both the technical direction and practical considerations of the system.

We also extend our gratitude to our internal supervisor, Varad Vishwarupe, for his constructive feedback during the development and reporting stages of the project.

Appendix

The project's Git repository consists of a monorepo and contains the frontend application, backend server, database migrations, and supporting configuration. The most relevant implementation directories are `frontend/src`, `server/internal`, and `server/migrations`.

Link: github.com/Contrabass26/hotdesk